

Sprint Documentation

Sprint #12	Start Date: 24/02/2026	Final Date: 09/03/2026
Projetc Name:	civitas-backend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
Gustavo Henrique Moreira Porto		
Felipe Pacheco Bianchini		
Giovanni da Silva Nascimento		

Sprint Backlog

Task #	Description	Start Date	Final date
050	Pesquisa por Paginação	24/02/2026	09/03/2026

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
050	O objetivo desta task é implementar um mecanismo de pesquisa com paginação no backend.	Gustavo Porto, Felipe Pacheco e Giovanni Nascimento	Completo	15 Horas	15 Horas

Sprint Description

Tarefa: Sprint 12 - Pesquisa por Paginação

Objetivo

O objetivo desta task é implementar um mecanismo de **pesquisa com paginação** no backend.

Devido ao grande volume de dados financeiros (despesas, empenhos, fornecedores, relatórios por período, etc.), é necessário garantir que as consultas retornem resultados de forma paginada, evitando:

- Sobrecarga no banco de dados
- Alto consumo de memória no servidor
- Respostas muito grandes na API
- Lentidão na interface do usuário

A paginação visa melhorar tanto o **desempenho do sistema** quanto a **experiência do usuário**, permitindo navegação eficiente entre grandes conjuntos de dados.

Critérios de Aceite

- Implementação de paginação nos endpoints de listagem (ex: despesas, fornecedores, secretarias).

Definição de parâmetros padrão (ex: page, limit ou size).

- Retorno estruturado contendo:
 - Lista de dados da página atual
 - Total de registros
 - Total de páginas
 - Página atual

Aplicação de ordenação segura e controlada.

Testes de performance comparando antes e depois da paginação.

Documentação da implementação no backend.

Observações

- Evitar uso de consultas sem limite (SELECT * sem LIMIT).
- Definir um limite máximo de registros por página para evitar abusos (ex: máximo de 100 registros por requisição).
- Avaliar uso de:
 - Paginação com LIMIT e OFFSET (modelo tradicional).
 - Paginação baseada em cursor (para grandes volumes de dados).
- Garantir compatibilidade com filtros combinados (por período, secretaria, fornecedor, status, etc.).
- Considerar impacto da paginação em consultas com grandes volumes históricos (anos fiscais anteriores).

Foco principal: **reduzir carga no sistema e melhorar a experiência do usuário ao navegar por grandes volumes de dados.**

Sprint Results

Durante a Sprint 12 implementamos um sistema de paginação nos endpoints de listagem da API, com o objetivo de melhorar a performance das consultas e evitar que grandes quantidades de dados sejam retornadas de uma só vez. A implementação foi feita utilizando LIMIT e OFFSET no repositório genérico, seguindo a arquitetura já existente do projeto (Controller → Service → Repository), sem criar novas camadas.

A paginação permite dividir os resultados em páginas menores, onde cada requisição retorna apenas uma parte dos registros. Para isso, os endpoints de listagem passaram a aceitar alguns parâmetros de consulta (query params).

O parâmetro `page` define qual página de resultados será retornada. Por padrão, o valor é 1, que corresponde à primeira página.

O parâmetro `size` define quantos registros serão retornados por página. O valor padrão é 20, e o máximo permitido é 100, para evitar sobrecarga no sistema.

Internamente, esses valores são usados para calcular o `LIMIT` e o `OFFSET` da consulta no banco de dados:

- `LIMIT` determina quantos registros serão retornados (baseado no `size`).
- `OFFSET` determina a partir de qual posição os registros começam a ser retornados (calculado com base na página).

Por exemplo:

- `page = 1` e `size = 20` → retorna os 20 primeiros registros
- `page = 2` e `size = 20` → pula os 20 primeiros registros e retorna os próximos 20

Além da paginação, os endpoints também permitem ordenar os resultados retornados. Isso é feito utilizando os parâmetros `sortBy` e `sortDirection`.

O parâmetro `sortBy` define qual campo da entidade será utilizado para ordenar os dados. Esse campo precisa existir na tabela ou no modelo da entidade. Por exemplo, em um endpoint de fornecedores, alguns valores possíveis poderiam ser:

- `NomeFantasia`
- `RazaoSocial`
- `DataCriacao`
- `Id`

Exemplo do que por: `sortBy=NomeFantasia`

Nesse caso, os fornecedores serão organizados de acordo com o valor do campo `NomeFantasia`.

Já o parâmetro `sortDirection` define a direção da ordenação, podendo ser:

- `asc` → ordem crescente
- `desc` → ordem decrescente

Exemplo: `sortDirection=asc`

Caso seja informado um campo inválido em `sortBy`, o sistema aplica automaticamente um fallback para a chave primária da entidade, garantindo que a ordenação continue funcionando.

A resposta da API foi padronizada para retornar não apenas os dados da página atual, mas também informações sobre a paginação, incluindo:

- `items`: registros retornados na página atual
- `totalRecords`: total de registros existentes
- `totalPages`: número total de páginas

- `currentPage`: página atual
- `pageSize`: quantidade de registros por página

Essa implementação foi aplicada aos endpoints de listagem que utilizam o serviço genérico, incluindo despesas, fornecedores, secretarias, usuários, auditorias, documentos, fluxos, orçamentos, instituições, tipos e unidades.

Além disso, foi criado um projeto de testes automatizados para validar o funcionamento da paginação, verificando cenários como retorno de payload paginado com metadados, limite máximo por página, ordenação segura e comparação entre leitura completa e leitura paginada no repositório.

Assinaturas